

Konzeption und Implementierung eines KI-gestützten GitLab-Assistenten zur Verbesserung der Qualität von Software-Tickets

6. Juli 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Theoretische und technische Grundlagen	6
2.1	Requirements Engineering und Anforderungen in Softwareentwicklungsprozessen	6
2.2	Tickets, User Stories und Bug Reports als Arbeitsartefakte	7
2.3	Qualität von Ticketinformationen	9
2.4	Generative KI und Large Language Models	10
2.5	LLM-basierte Agenten, Werkzeugnutzung und Opencode	11
2.6	GitLab, Webhooks und technische Integrationsplattform	12
2.7	Risiken und Validierungsbedarf bei generativer KI und Webhooks	14
2.8	Zwischenfazit und Anforderungen an Dev-Assist	16
3	Anforderungsanalyse	17
3.1	Vorgehen und Grundlagen der Anforderungsanalyse	17
3.2	Beschreibung des bisherigen Prozesses	18
3.3	Zielgruppen des Systems	19
3.4	Fachliche Anforderungen an die Ticketqualität	20
3.5	Funktionale Anforderungen	22
3.6	Nicht-funktionale Anforderungen	23
3.7	Abgrenzung der Anforderungen	25
3.8	Priorisierung der Anforderungen	25
3.9	Zwischenfazit	26
	Quellen	28

1 Einleitung

In modernen Softwareentwicklungsprozessen werden Anforderungen, Fehlerberichte und ergänzende Hinweise häufig in Ticket- oder Issue-Tracking-Systemen dokumentiert. Diese Artefakte sind nicht nur technische Aufgabenlisten, sondern Koordinationspunkte zwischen meldenden Personen, Produktverantwortlichen und Entwicklerinnen und Entwicklern. Studien zu Bug Reports zeigen, dass Bug-Tracking-Systeme eine zentrale Rolle in der Zusammenarbeit zwischen Nutzenden und Entwicklungsteams übernehmen und dass Rückfragen in Tickets häufig dazu dienen, fehlende Informationen nachzuliefern (vgl. Breu et al., 2010, S. 301 und S. 303). Gleichzeitig unterscheiden sich die Informationen, die meldende Personen bereitstellen, deutlich von den Informationen, die Entwicklerinnen und Entwickler für eine schnelle Bearbeitung benötigen. Besonders Reproduktionsschritte, Stack Traces und Testfälle gelten als hilfreich, sind für meldende Personen aber schwer bereitzustellen (vgl. Bettenburg et al., 2008, S. 308 f.).

Diese Diskrepanz betrifft nicht nur klassische Fehlerberichte, sondern auch Anforderungen in agilen Entwicklungsprozessen. Anforderungen werden dort häufig in knappen, natürlichsprachlichen Formen wie User Stories oder Tickets erfasst. Ihre Qualität hängt davon ab, ob Ziel, Kontext, erwartetes Verhalten, Randbedingungen und Akzeptanzkriterien so beschrieben sind, dass sie von Entwicklungsteams verstanden, umgesetzt und überprüft werden können. Lucassen et al. beschreiben User Stories als weit verbreitete Notation im agilen Requirements Engineering, weisen jedoch zugleich darauf hin, dass sie in der Praxis häufig Qualitätsdefekte aufweisen. Das Quality User Story Framework fasst diese Qualität unter anderem über syntaktische, semantische und pragmatische Kriterien (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Für Fehlerberichte zeigen Chaparro et al. ähnlich, dass Beschreibungen für die Bearbeitung insbesondere beobachtetes Verhalten, erwartetes Verhalten und Schritte zur Reproduktion enthalten sollten, solche Informationen jedoch oft fehlen oder nur unvollständig vorliegen (vgl. Chaparro et al., 2017, S. 396 f.).

Damit entsteht ein praktisches Problem an der Schnittstelle zwischen Kommunikation und Entwicklung: Ein Ticket kann fachlich relevant sein, aber dennoch nicht in einer Form vorliegen, die unmittelbar für Planung, Implementierung und Validierung geeignet ist. Entwicklungsteams müssen dann fehlende Informationen nachfragen, vorhandene Kommentare interpretieren, Screenshots oder Anhänge einordnen und aus verstreutem Kontext eine umsetzbare Anforderung ableiten. Wissenschaftliche Arbeiten und Werkzeuge zeigen, dass automatisierte Unterstützung bei der Bewertung und Strukturierung solcher Informationen grundsätzlich möglich ist. CUEZILLA bewertet die Qualität von Bug Reports und schlägt ergänzende Informationen vor, AQUASA erkennt Qualitätsdefekte in User Stories und DeMiBuD identifiziert insbesondere fehlendes erwartetes Verhalten sowie fehlende Reproduktionsschritte in Bug Descriptions. Diese Ansätze leisten damit jeweils unterschiedliche Beiträge zur automatisierten Qualitätsunterstützung. Für diese Arbeit bleibt darüber hinaus die Frage offen, wie eine

solche Unterstützung in einen konkreten, laufenden Entwicklungsworkflow eingebunden werden kann (vgl. Bettenburg et al., 2008, S. 308 f.; Lucassen et al., 2016, S. 383 f. und S. 386 f.; Chaparro et al., 2017, S. 401 ff.).

Generative KI und große Sprachmodelle eröffnen für diese Aufgabe neue Möglichkeiten. Brown et al. zeigen am Beispiel GPT-3, dass ein großes Sprachmodell Aufgaben ohne Gewichtsaktualisierung oder aufgabenspezifisches Fine-Tuning bearbeiten kann, wenn Aufgabenbeschreibung und Beispiele im Prompt bereitgestellt werden (vgl. Brown et al., 2020, S. 1 und S. 3). Daraus lässt sich für diese Arbeit ableiten, dass große Sprachmodelle grundsätzlich dafür geeignet sein können, natürlichsprachliche Ticketinformationen flexibel zu verarbeiten. Für die Anforderungsaufbereitung ist insbesondere relevant, dass solche Modelle nicht nur Texte zusammenfassen, sondern Inhalte klassifizieren, fehlende Informationen identifizieren, Rückfragen formulieren und bestehende Beschreibungen in strukturierte Formate überführen können. Damit eignen sie sich prinzipiell als Assistenzsysteme, die unstrukturierte Ticketinformationen analysieren und daraus einen konsistenteren Vorschlag für Titel, Beschreibung, Akzeptanzkriterien oder offene Fragen ableiten.

Über die reine Textgenerierung hinaus werden große Sprachmodelle zunehmend als handlungsfähige Komponenten in Werkzeugketten betrachtet. Der ReAct-Ansatz verbindet sprachliche Zwischenschritte mit aufgabenspezifischen Aktionen, sodass ein Modell Informationen aus externen Quellen einbeziehen und sein Vorgehen nachvollziehbarer machen kann (vgl. Yao et al., 2023, S. 1 f.). Toolformer zeigt ergänzend, dass Sprachmodelle den Einsatz externer Werkzeuge über APIs als Teil der Problemlösung nutzen können (vgl. Schick et al., 2023, S. 1). Für einen Developer Assistant ist dieser Gedanke zentral: Das System soll nicht isoliert einen Text erzeugen, sondern auf Ereignisse in GitLab reagieren, Ticketdaten und Kommentare abrufen, eine Analyse anstoßen und das Ergebnis wieder im Entwicklungskontext bereitstellen. GitLab-Webhooks ermöglichen hierfür eine ereignisbasierte Integration, bei der externe Anwendungen über Änderungen wie Issue-Updates oder neue Kommentare informiert werden können (vgl. GitLab Docs, o. J., o. S.).

Der Einsatz generativer KI in einem solchen Prozess ist jedoch nicht ohne Risiken. Generative KI kann überzeugend formulierte, aber falsche oder nicht belegte Inhalte erzeugen; NIST beschreibt dieses Risiko als Confabulation beziehungsweise Hallucination (vgl. NIST, 2024, S. 4 und S. 6). Für ein System, das Anforderungen für Entwicklerinnen und Entwickler aufbereitet, wäre dies besonders problematisch, weil unzutreffende Annahmen direkt in Umsetzungsvorschläge, Akzeptanzkriterien oder technische Entscheidungen einfließen könnten. Hinzu kommt, dass LLM-integrierte Anwendungen für Angriffe wie Prompt Injection anfällig sein können, wenn unkontrollierte Eingaben das Verhalten des Systems beeinflussen (vgl. NIST, 2024, S. 11). Ein Developer Assistant muss daher nicht nur gute Vorschläge erzeugen, sondern seine Ausgaben technisch begrenzen, validieren und in einen kontrollierten Freigabeprozess einbetten.

Vor diesem Hintergrund befasst sich die vorliegende Bachelorarbeit mit der Konzeption und Entwicklung eines KI-gestützten Developer Assistants zur automatisierten Anforderungsaufbereitung in Softwareentwicklungsprozessen. Der prototypische Assistant, im Folgenden Dev-Assist genannt, reagiert über GitLab-Webhooks auf neu erstellte oder aktualisierte Tickets sowie auf relevante Kommentare. Anschließend analysiert er Titel, Beschreibung, Diskussion und weitere Ticketinformationen mithilfe genera-

tiver KI und überführt den vorhandenen Kontext in ein standardisiertes Format. Je nach Informationslage soll das System entweder gezielte Rückfragen stellen oder einen strukturierten Vorschlag für die weitere Bearbeitung erzeugen. Der aufbereitete Kontext dient dabei als Grundlage für nachgelagerte Entwicklungsprozesse und soll die Übergabe zwischen Anforderung, Umsetzung und Validierung verbessern.

Die Arbeit untersucht damit zwei eng verbundene Fragestellungen. Erstens ist zu klären, welche Informationen ein GitLab-Ticket enthalten muss, damit es von Entwicklerinnen und Entwicklern nachvollziehbar umgesetzt und überprüft werden kann. Zweitens ist zu untersuchen, wie ein KI-gestützter Assistant technisch so in einen GitLab-basierten Entwicklungsworkflow integriert werden kann, dass Analyse, Rückfragen, Vorschläge und Freigabe kontrolliert ablaufen. Dazu gehören neben der Webhook-Verarbeitung und der Kommunikation mit GitLab-APIs auch die Strukturierung der KI-Ausgabe, die Erkennung ungültiger Antworten, der Umgang mit Bild- und Kommentar-Kontext sowie die Frage, welche zusätzlichen Skills oder Validierungsschritte erforderlich sind, um eine möglichst fehlerarme Umsetzung zu unterstützen.

Ziel der Arbeit ist nicht, menschliche Abstimmung in der Anforderungsanalyse zu ersetzen. Vielmehr soll untersucht werden, wie ein Assistenzsystem wiederkehrende Aufbereitungsaufgaben übernehmen und Entwicklungsteams dadurch entlasten kann. Der Beitrag der Arbeit liegt deshalb in einem umgesetzten Prototyp sowie in der Bewertung, welche Architektur- und Validierungsentscheidungen notwendig sind, damit generative KI in einem realistischen Ticket-Workflow praktisch nutzbar wird. Die folgenden Kapitel führen zunächst in die Grundlagen von Requirements Engineering, Ticketqualität, generativer KI und agentenbasierten Werkzeugketten ein. Anschließend werden Konzeption, Implementierung und Validierung des Dev-Assist-Prototyps beschrieben. Abschließend werden die Ergebnisse eingeordnet, Grenzen des Ansatzes diskutiert und mögliche Weiterentwicklungen aufgezeigt.

2 Theoretische und technische Grundlagen

Dieses Kapitel schafft die fachliche und technische Grundlage für die Konzeption des KI-gestützten GitLab-Assistenten. Im Mittelpunkt stehen drei Fragen: Erstens ist zu klären, welche Eigenschaften Software-Tickets besitzen müssen, damit sie für Entwicklungsteams nutzbar sind. Zweitens ist zu beschreiben, welche Rolle GitLab-Issues, Kommentare und Webhooks in einem ereignisbasierten Entwicklungsworkflow einnehmen. Drittens werden Large Language Models, KI-Agenten, Opencode sowie die verwendete Node.js-/TypeScript-/Express-Umgebung eingeordnet, damit die spätere Architektur des Prototyps nachvollziehbar begründet werden kann.

2.1 Requirements Engineering und Anforderungen in Softwareentwicklungsprozessen

Requirements Engineering bezeichnet den Teil der Softwareentwicklung, in dem Anforderungen erhoben, analysiert, dokumentiert, geprüft und verwaltet werden. Anforderungen bilden die Grundlage dafür, welche Funktionalität ein System bereitstellen soll, welche Qualitätsmerkmale es erfüllen muss und unter welchen Randbedingungen es betrieben wird. Damit sind Anforderungen nicht nur fachliche Beschreibungen, sondern auch verbindliche Kommunikationsartefakte zwischen Auftraggebern, Nutzenden, Produktverantwortlichen, Entwicklung und Qualitätssicherung.

Die Bedeutung dieses Tätigkeitsbereichs zeigt sich auch in normativen Grundlagen. Die öffentlich zugängliche Katalogseite zu ISO/IEC/IEEE 29148 beschreibt Requirements Engineering als Prozessbereich, der Anforderungen für Systeme und Softwareprodukte über den Lebenszyklus hinweg hervorbringt, zugehörige Informationsprodukte definiert und Vorgaben für deren Inhalte und Formate bereitstellt (vgl. ISO/IEC/IEEE, 2018, o. S.). Für diese Arbeit wird daraus nur die allgemeine Einordnung des Standards herangezogen; detaillierte normative Abschnittsvorgaben des vollständigen Standards werden nicht vorausgesetzt. Besonders relevant ist daran, dass Anforderungen nicht isoliert als einzelne Texte betrachtet werden können. Sie stehen in einem Prozesszusammenhang: Eine Anforderung muss verstanden, umgesetzt, überprüft und bei Änderungen nachvollziehbar angepasst werden können.

Eine gute Anforderung sollte daher mehrere Qualitätsmerkmale erfüllen. Sie muss verständlich formuliert sein, damit alle Beteiligten denselben Sachverhalt meinen. Sie muss eindeutig genug sein, damit Entwicklungsteams nicht mehrere widersprüchliche Interpretationen ableiten. Sie muss prüfbar sein, damit Umsetzung und Akzeptanz nicht nur subjektiv bewertet werden. Außerdem muss sie ausreichend vollständig sein, damit

zentrale fachliche und technische Informationen nicht erst während der Implementierung rekonstruiert werden müssen. Die öffentlich zugängliche Katalogseite zu ISO/IEC 25010 ordnet Qualitätsmodelle unter anderem als Hilfsmittel ein, um Anforderungen zu spezifizieren, ihre Vollständigkeit zu validieren, Testziele abzuleiten und Akzeptanzkriterien zu bestimmen (vgl. ISO/IEC, 2023, o. S.). Auch wenn ISO/IEC 25010 primär ein Produktqualitätsmodell beschreibt, ist der Bezug zur Anforderungsarbeit deutlich: Qualität muss so beschrieben werden, dass sie später bewertet werden kann.

In agilen Entwicklungsprozessen werden Anforderungen häufig nicht in umfangreichen Spezifikationsdokumenten, sondern in kleineren Artefakten wie User Stories, Tasks oder Tickets festgehalten. Diese Form reduziert den Dokumentationsaufwand und unterstützt iterative Entwicklung, erhöht aber zugleich die Abhängigkeit von klarer Kommunikation. Inayat et al. beschreiben agile Requirements Engineering als eine stärker kollaborative Form der Anforderungsarbeit, in der Planung, Ausführung und Begründung von Anforderungen eng mit Kommunikation im Team verbunden sind (vgl. Inayat et al., 2015, S. 915 ff.). Für den Kontext dieser Arbeit bedeutet das: Wenn Anforderungen in GitLab-Issues und Kommentaren entstehen, dann entscheidet die Qualität dieser Artefakte darüber, ob ein Entwicklungsteam ohne zusätzliche Reibung weiterarbeiten kann.

Lucassen et al. zeigen am Beispiel von User Stories, dass solche kurzen, natürlichsprachlichen Anforderungen in der Praxis weit verbreitet sind, aber häufig Qualitätsdefekte aufweisen. Das Quality User Story Framework unterscheidet Qualitätskriterien unter anderem in syntaktische, semantische und pragmatische Aspekte (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Syntaktische Qualität betrifft dabei die Form einer Anforderung, semantische Qualität die inhaltliche Bedeutung und Konsistenz, pragmatische Qualität die Nutzbarkeit für die Beteiligten. Diese Einordnung lässt sich auf Software-Tickets übertragen: Ein Ticket kann sprachlich lesbar sein, aber dennoch für Entwicklerinnen und Entwickler unzureichend bleiben, wenn erwartetes Verhalten, Kontext, Akzeptanzkriterien oder technische Randbedingungen fehlen.

Für die vorliegende Arbeit ergibt sich daraus ein grundlegendes Verständnis: Ein GitLab-Issue ist nicht automatisch eine gute Anforderung. Es ist zunächst ein Container für Titel, Beschreibung, Kommentare, Metadaten und Anhänge. Erst wenn diese Informationen so strukturiert sind, dass sie fachlich verstanden, technisch umgesetzt und später überprüft werden können, erfüllt das Ticket die Rolle einer verwertbaren Anforderung. Der geplante Developer Assistant setzt genau an dieser Stelle an: Er soll vorhandene Ticketinformationen analysieren, fehlende Bestandteile sichtbar machen und vorhandenen Kontext in eine besser nutzbare Form überführen.

2.2 Tickets, User Stories und Bug Reports als Arbeitsartefakte

Software-Tickets dienen in Entwicklungsprozessen als Arbeitsartefakte. Sie beschreiben Aufgaben, Fehler, Änderungswünsche oder fachliche Anforderungen und verbinden diese Informationen mit Status, Zuständigkeiten, Kommentaren und weiteren Metadaten. In GitLab werden solche Artefakte typischerweise als Issues geführt. Sie können eine

kurze Problem- oder Anforderungsbeschreibung enthalten, durch Kommentare ergänzt werden und im Verlauf der Bearbeitung aktualisiert werden. Damit sind Tickets sowohl Dokumentationsform als auch Prozessschnittstelle.

Zwischen User Stories, Bug Reports und allgemeinen Tickets bestehen Unterschiede, die für die Arbeit wichtig sind. Eine User Story beschreibt meist einen gewünschten fachlichen Nutzen aus Sicht einer Rolle, häufig in der Form "Als ... möchte ich ..., damit ...". Ein Bug Report beschreibt dagegen eine Abweichung zwischen beobachtetem und erwartetem Systemverhalten. Ein allgemeines Ticket kann beide Formen enthalten oder auch eine technische Aufgabe beschreiben. Gemeinsam ist diesen Artefakten, dass sie in natürlicher Sprache formuliert werden und häufig von Personen mit unterschiedlichem technischen Hintergrund erstellt oder ergänzt werden. Dadurch entstehen Informationslücken und Interpretationsspielräume.

Bug Reports sind in der Forschung gut untersucht und eignen sich deshalb als Referenz für Ticketqualität. Bettenburg et al. zeigen, dass Bug Reports für Entwicklerinnen und Entwickler wichtige Informationen enthalten, aber stark in ihrer Qualität variieren. Besonders deutlich ist die Diskrepanz zwischen Informationen, die Entwicklungsteams benötigen, und Informationen, die meldende Personen bereitstellen. In der Studie wurden Reproduktionsschritte, Stack Traces und Testfälle von Entwicklerseite als hilfreich bewertet, während genau diese Angaben für Reporterinnen und Reporter schwer bereitzustellen sind (vgl. Bettenburg et al., 2008, S. 308 f.). Diese Erkenntnis ist auf GitLab-Issues übertragbar: Auch dort können meldende Personen ein reales Problem beschreiben, ohne alle Informationen zu liefern, die für die Umsetzung oder Fehleranalyse erforderlich sind.

Breu et al. betonen, dass Bug-Tracking-Systeme eine zentrale Rolle in der Zusammenarbeit zwischen Nutzenden und Entwicklungsteams spielen. Ihre Analyse von Fragen in 600 Bug Reports aus Mozilla- und Eclipse-Projekten zeigt, dass die aktive Beteiligung der Nutzenden über das reine Melden eines Fehlers hinausgeht (vgl. Breu et al., 2010, S. 301). Besonders relevant ist die Kategorie "Missing Information": Entwicklerinnen und Entwickler fragen unter anderem nach Reproduktionsschritten, Build-Nummern, Betriebssystemen, Testfällen, Beispielen, Programmausgaben und Screenshots (vgl. Breu et al., 2010, S. 303). Ein Ticket ist damit nicht nur ein statischer Text, sondern häufig ein Gesprächsverlauf, in dem fehlende Informationen nach und nach ergänzt werden.

Für einen GitLab-Assistenten ist dieser Gesprächscharakter zentral. Wenn ein Issue erstellt wird, liegt der vollständige Entwicklungskontext oft noch nicht vor. Kommentare können zusätzliche Hinweise, Rückfragen, Screenshots oder Entscheidungen enthalten. Dadurch entsteht ein verteilter Kontext: Ein Teil steht im Titel, ein Teil in der Beschreibung, ein Teil in späteren Kommentaren und ein weiterer Teil möglicherweise in Bildern oder Links. Ein Assistenzsystem, das Ticketqualität verbessern soll, darf daher nicht nur die ursprüngliche Beschreibung betrachten. Es muss den gesamten verfügbaren Ticketkontext einbeziehen.

Lucassen et al. ergänzen diese Perspektive für agile Anforderungen. User Stories sind zwar als kurze Anforderungen praktikabel, ihre Qualität hängt jedoch davon ab, ob sie inhaltlich und sprachlich ausreichend präzise sind (vgl. Lucassen et al., 2016, S. 383 f.). Übertragen auf GitLab-Issues bedeutet dies: Kürze ist kein Qualitätsmerkmal an sich. Ein knappes Ticket kann gut sein, wenn es Ziel, Kontext, erwartetes Ergebnis und

Prüfkriterien eindeutig beschreibt. Es kann aber auch unbrauchbar sein, wenn es nur eine vage Idee enthält. Der Developer Assistant muss deshalb nicht nur "mehr Text" erzeugen, sondern relevante Informationen strukturieren und Lücken gezielt benennen.

2.3 Qualität von Ticketinformationen

Die Qualität von Ticketinformationen lässt sich aus mehreren Perspektiven betrachten. Aus fachlicher Sicht muss ein Ticket beschreiben, welches Problem gelöst oder welche Funktion umgesetzt werden soll. Aus technischer Sicht müssen Informationen vorhanden sein, die eine Umsetzung oder Analyse ermöglichen. Aus Sicht der Qualitätssicherung müssen erwartetes Verhalten und Akzeptanzkriterien prüfbar sein. Aus Sicht des Projektmanagements sollte erkennbar sein, welche Aufgabe zu erledigen ist, welche Abhängigkeiten bestehen und wann die Bearbeitung abgeschlossen werden kann.

Für Bug Reports identifizieren Chaparro et al. drei besonders wichtige Informationstypen: Observed Behavior, Steps to Reproduce und Expected Behavior. Ein wirksamer Bug Report sollte also beschreiben, was tatsächlich passiert ist, welche Schritte zu diesem Verhalten führen und welches Verhalten stattdessen erwartet wurde (vgl. Chaparro et al., 2017, S. 396 f.). Diese Struktur ist auch für allgemeine Tickets hilfreich. Bei Fehlern ist sie unmittelbar anwendbar. Bei neuen Anforderungen entspricht sie sinngemäß der Frage, welcher Ist-Zustand oder Bedarf besteht, welches Zielverhalten erreicht werden soll und anhand welcher Kriterien die Umsetzung nachvollzogen werden kann.

Chaparro et al. zeigen außerdem, dass diese Informationen häufig fehlen. Sie entwickelten mit DeMIBuD einen Ansatz, der fehlendes Expected Behavior und fehlende Steps to Reproduce in Bug Descriptions erkennen kann (vgl. Chaparro et al., 2017, S. 397 und S. 401 ff.). Für die vorliegende Arbeit ist weniger die konkrete technische Umsetzung von DeMIBuD entscheidend, sondern das Grundprinzip: Ticketqualität lässt sich teilweise automatisiert prüfen, wenn relevante Informationsbestandteile bekannt sind. Dev-Assist folgt einem ähnlichen Ziel, soll jedoch nicht nur fehlende Bestandteile erkennen, sondern im GitLab-Workflow Rückfragen oder strukturierte Vorschläge erzeugen.

Die Qualität von Reproduktionsschritten verdient besondere Aufmerksamkeit. Chaparro et al. untersuchen in einer späteren Arbeit die Qualität der Steps to Reproduce und beschreiben, dass schlechte Reproduktionsschritte zu zusätzlichem manuellem Aufwand bei Triaging und Fehlerbehebung führen. Ihr Ansatz Euler identifiziert und bewertet Reproduktionsschritte und gibt Rückmeldung zur Verbesserung des Bug Reports (vgl. Chaparro et al., 2019, S. 152). Daraus folgt für Software-Tickets: Nicht nur das Vorhandensein eines Abschnitts ist relevant, sondern auch dessen Nutzbarkeit. Reproduktionsschritte müssen konkret, nachvollziehbar und in einer plausiblen Reihenfolge beschrieben sein.

Song und Chaparro stellen mit BEE ein Werkzeug vor, das Bug Reports automatisch analysiert und Rückmeldung zu Observed Behavior, Expected Behavior und Steps to Reproduce gibt. BEE erkennt unter anderem, ob ein Issue einen Bug, eine Erweiterung oder eine Frage beschreibt, markiert die Struktur innerhalb einer Bug Description und erkennt fehlende Elemente (vgl. Song/Chaparro, 2020, S. 1551 f.). Diese Arbeit zeigt,

dass strukturierte Analyse von Tickettexten nicht nur theoretisch möglich ist, sondern praktisch als Werkzeug in Entwicklungsplattformen eingebunden werden kann. Für Dev-Assist ist daran besonders relevant, dass Ticketanalyse nicht als isolierte Textklassifikation verstanden wird, sondern als Unterstützung für Reporterinnen, Reporter und Entwicklungsteams.

Aus diesen Arbeiten lassen sich konkrete Qualitätsdimensionen für GitLab-Tickets ableiten. Ein Ticket sollte einen aussagekräftigen Titel besitzen, der den Gegenstand knapp benennt. Die Beschreibung sollte Problem oder Ziel, Kontext, erwartetes Ergebnis, relevante Schritte, technische Randbedingungen und Akzeptanzkriterien enthalten. Kommentare sollten nicht nur lose Diskussionen bleiben, sondern in den Ticketkontext einfließen, wenn sie entscheidungsrelevante Informationen enthalten. Anhänge und Screenshots sollten so referenziert sein, dass ihre Bedeutung erkennbar ist. Außerdem sollte ein Ticket zwischen gesicherten Informationen, Annahmen und offenen Fragen unterscheiden.

Gerade an dieser Unterscheidung setzt ein KI-gestützter Assistent an. Wenn Informationen fehlen, sollte er nicht frei spekulieren, sondern Rückfragen formulieren. Wenn genügend Kontext vorhanden ist, kann er einen strukturierten Vorschlag erzeugen. Wenn Kommentare widersprüchlich sind, sollte dies als Unsicherheit sichtbar werden. Ticketqualität entsteht also nicht allein durch sprachliche Glättung, sondern durch die kontrollierte Verarbeitung von Kontext.

2.4 Generative KI und Large Language Models

Large Language Models sind Sprachmodelle, die auf sehr großen Textmengen trainiert werden und natürliche Sprache verstehen, fortsetzen und erzeugen können. Die Grundlage vieler moderner Modelle bildet die Transformer-Architektur. Vaswani et al. schlagen den Transformer als Architektur vor, die auf Attention-Mechanismen basiert und auf rekurrente oder konvolutionale Strukturen verzichtet (vgl. Vaswani et al., 2017, S. 1 f.). Für diese Arbeit ist keine detaillierte mathematische Beschreibung der Transformer-Architektur erforderlich. Entscheidend ist, dass Transformer-basierte Modelle lange Eingaben verarbeiten, Beziehungen zwischen Textbestandteilen gewichten und dadurch komplexe Sprachaufgaben lösen können.

Brown et al. zeigen am Beispiel GPT-3, dass große Sprachmodelle Aufgaben aus Instruktionen und wenigen Beispielen im Prompt bearbeiten können, ohne dass für jede Aufgabe ein eigenes Fine-Tuning oder eine Gewichtsaktualisierung erforderlich ist. Sie bezeichnen diesen Mechanismus als In-Context Learning (vgl. Brown et al., 2020, S. 1 und S. 3). Für Dev-Assist ist das relevant, weil Ticketanalyse eine variable Aufgabe ist. Jedes Issue enthält andere Formulierungen, andere Kommentare, andere fachliche Inhalte und unterschiedliche Informationslücken. Ein starres Regelwerk wäre schwer zu pflegen. Ein LLM kann dagegen auf Basis eines Prompts angewiesen werden, Ticketinformationen zu prüfen, Rückfragen zu stellen oder einen Vorschlag in einem standardisierten Format zu erzeugen.

Zhao et al. beschreiben Large Language Models als Ergebnis einer Entwicklung von klassischen Sprachmodellen zu großen, vortrainierten Modellen, die durch Skalierung neue

Fähigkeiten zeigen können. Der Survey ordnet LLMs unter anderem nach Pre-Training, Anpassung, Nutzung und Evaluation ein (vgl. Zhao et al., 2023, S. 1). Für diese Arbeit ist vor allem die Nutzungsseite relevant. Ein LLM wird nicht selbst trainiert, sondern über eine Agentenschnittstelle in einen bestehenden Workflow eingebunden. Die Qualität des Ergebnisses hängt deshalb stark von Promptgestaltung, Kontextauswahl, Ausgabeformat und Validierung ab.

Prompt Engineering bezeichnet die Gestaltung von Eingaben, mit denen das Modell zu einer gewünschten Aufgabe angeleitet wird. Zhao et al. beschreiben Prompt Engineering als manuelles Erstellen geeigneter Prompts, um Fähigkeiten von LLMs für bestimmte Aufgaben hervorzurufen (vgl. Zhao et al., 2023, S. 44). Im Kontext dieser Arbeit bedeutet das: Der Prompt muss dem Modell nicht nur sagen, dass ein Ticket analysiert werden soll. Er muss auch definieren, welche Informationen relevant sind, welche Ausgabeform erwartet wird, wann Rückfragen gestellt werden sollen und wie mit Bildern oder Kommentaren umzugehen ist.

Gleichzeitig darf die Leistungsfähigkeit von LLMs nicht überschätzt werden. Sie erzeugen plausible Sprache, besitzen aber kein garantiert korrektes Verständnis des tatsächlichen Systems. Sie können aus unvollständigem Kontext Annahmen ableiten, die fachlich nicht belegt sind. Deshalb ist in dieser Arbeit wichtig, LLMs nicht als autonome Entscheider zu behandeln. Sie dienen als Assistenzkomponente, die Vorschläge formuliert und Informationslücken sichtbar macht. Die Verantwortung für endgültige Ticketänderungen bleibt beim Menschen beziehungsweise beim kontrollierten Publish-Prozess.

Für die spätere Implementierung folgt daraus eine technische Anforderung: Die Ausgabe des Modells muss maschinenlesbar und prüfbar sein. Wenn Dev-Assist eine Entscheidung zwischen Rückfrage und Vorschlag treffen soll, reicht ein freier Fließtext nicht aus. Die Antwort muss ein definiertes Format besitzen, etwa mit Feldern für `hasQuestions`, `questions`, `proposedTitle` und `proposedDescription`. Nur so kann das System die Antwort weiterverarbeiten, Fehler erkennen und robuste Tests schreiben.

2.5 LLM-basierte Agenten, Werkzeugnutzung und Opencode

Ein LLM-basierter Agent unterscheidet sich von einem einfachen Chatbot dadurch, dass er nicht nur eine Antwort auf eine Nutzereingabe erzeugt, sondern in einen Handlungskontext eingebettet ist. Er kann Informationen aufnehmen, Zwischenschritte ausführen, Werkzeuge nutzen und Ergebnisse wieder in eine Umgebung zurückspielen. Xi et al. beschreiben LLM-basierte Agenten als Systeme, in denen große Sprachmodelle als Grundlage für Agenten dienen. Ihr allgemeines Framework besteht aus den Komponenten Brain, Perception und Action (vgl. Xi et al., 2023, S. 1). Übertragen auf Dev-Assist bedeutet dies: Das Modell verarbeitet den Ticketkontext, nimmt GitLab-Issue-Daten und Kommentare als Wahrnehmung auf und erzeugt als Handlung Rückfragen oder strukturierte Vorschläge.

Yao et al. verbinden im ReAct-Ansatz sprachliches Schlussfolgern mit Aktionen. Das Modell erzeugt Reasoning Traces und aufgabenspezifische Aktionen in verschränkter

Form, wodurch es Informationen aus externen Quellen oder Umgebungen einbeziehen kann (vgl. Yao et al., 2023, S. 1 f.). Für die vorliegende Arbeit ist daran nicht die exakte ReAct-Implementierung entscheidend, sondern das Prinzip: Ein KI-System im Entwicklungsworkflow sollte Kontext aus der Umgebung einbeziehen und nicht nur auf eine isolierte Texteingabe reagieren. Dev-Assist ruft deshalb Issue-Daten, Kommentare und Bildreferenzen ab, bevor eine Analyse gestartet wird.

Schick et al. zeigen mit Toolformer, dass Sprachmodelle lernen können, externe Werkzeuge über einfache APIs einzusetzen. Toolformer entscheidet unter anderem, welche APIs aufgerufen werden, wann sie aufgerufen werden, welche Argumente verwendet werden und wie Ergebnisse in die weitere Vorhersage einfließen (vgl. Schick et al., 2023, S. 1). Auch hier steht für diese Arbeit nicht das konkrete Verfahren im Vordergrund, sondern der Architekturgedanke: Leistungsfähige LLM-Anwendungen entstehen nicht nur durch Modellantworten, sondern durch die Kombination aus Modell, Werkzeugen, Datenquellen und kontrollierten Schnittstellen.

OpenCode wird in dieser Arbeit als Agentenschnittstelle eingesetzt. Die offizielle Dokumentation beschreibt Agents als spezialisierte KI-Assistenten, die für bestimmte Aufgaben und Workflows konfiguriert werden können, einschließlich eigener Prompts, Modelle und Tool-Zugriffe (vgl. OpenCode Docs, o. J.a, Abschnitt "Agents"). Die SDK-Dokumentation beschreibt den JS/TS-Client als typischere Schnittstelle, mit der Integrationen gebaut und OpenCode programmatisch gesteuert werden können (vgl. OpenCode Docs, o. J.b, Abschnitt "SDK"). Für Dev-Assist ist diese Eigenschaft zentral, weil der GitLab-Webhook nicht interaktiv im Terminal arbeitet, sondern serverseitig eine Agentensession starten, einen Prompt senden und die Antwort weiterverarbeiten muss.

Die OpenCode-Tool-Dokumentation beschreibt zudem, dass Tools dem LLM Aktionen in der Codebasis ermöglichen und dass Tool-Verhalten über Berechtigungen gesteuert werden kann (vgl. OpenCode Docs, o. J.c, Abschnitt "Tools"). Im Rahmen dieser Arbeit wird diese Werkzeugperspektive vor allem konzeptionell relevant: Ein Agent braucht begrenzte, nachvollziehbare Fähigkeiten. Für den Dev-Assist-Prototyp bedeutet das, dass die eigentliche GitLab-Verarbeitung nicht beliebig durch das Modell erfolgt. Stattdessen ruft die Anwendung kontrolliert Daten ab, übergibt einen definierten Kontext an den Agenten und verarbeitet die Antwort nach festen Regeln.

Damit ergibt sich eine klare Rollenverteilung. GitLab liefert den Ereignis- und Ticketkontext. Die Anwendung entscheidet, ob ein Ereignis relevant ist. OpenCode stellt die Agentenschnittstelle bereit. Das LLM erzeugt Rückfragen oder Vorschläge. Die Anwendung validiert die Antwort und veröffentlicht sie kontrolliert als Kommentar oder übernimmt sie nach einem Publish-Kommando. Diese Trennung ist wichtig, weil sie den Handlungsspielraum des Modells begrenzt und die spätere Testbarkeit erhöht.

2.6 GitLab, Webhooks und technische Integrationsplattform

GitLab ist im Kontext dieser Arbeit die zentrale Entwicklungs- und Integrationsplattform. Issues bilden die primären Ticketartefakte. Kommentare beziehungsweise Notes

ergänzen den Verlauf und enthalten Rückfragen, Hinweise oder Freigabekommandos. Discussions strukturieren Gesprächsverläufe in Threads. Webhooks verbinden GitLab mit externen Systemen, indem sie Ereignisse als HTTP-Anfragen an konfigurierte Endpunkte senden.

Die GitLab-Dokumentation beschreibt Webhooks als Mechanismus, der GitLab mit anderen Tools und Systemen über Echtzeitbenachrichtigungen verbindet. Bei wichtigen Ereignissen sendet GitLab Informationen an externe Anwendungen, sodass Automatisierungsworkflows auf Merge Requests, Code Pushes oder Issue-Updates reagieren können (vgl. GitLab Docs, o. J.a, Abschnitt "Webhook events"). Die Webhook-Events-Dokumentation ergänzt, dass Webhooks HTTP-POST-Anfragen mit detaillierten Informationen an konfigurierte Endpunkte senden, wenn bestimmte Ereignisse eintreten. Dazu gehören unter anderem Kommentare auf Issues (vgl. GitLab Docs, o. J.b, Abschnitt "Comment events").

Für Dev-Assist sind besonders Issue- und Note-Events relevant. Ein Issue-Event kann ausgelöst werden, wenn ein Issue erstellt oder aktualisiert wird. Ein Note-Event kann entstehen, wenn ein Kommentar hinzugefügt wird. Dadurch kann der Assistent auf zwei Arten aktiviert werden: durch ein neu erstelltes oder aktualisiertes Ticket, das `@dev-assist` erwähnt, oder durch einen Kommentar, der den Assistenten anspricht. Diese Ereignisorientierung passt zur Zielsetzung, weil der Assistent nicht dauerhaft pollend nach Änderungen suchen muss, sondern auf konkrete GitLab-Ereignisse reagieren kann.

Die GitLab Issues API zeigt, dass Issues unter anderem Titel, Beschreibung, Status, Autorenschaft, Labels, Assignees und weitere Metadaten enthalten können. Die Beschreibung eines Issues ist ein eigenes Feld und kann aktualisiert werden (vgl. GitLab Docs, o. J.c, o. S.). Die Notes API ermöglicht das Abrufen von Kommentaren zu einem Issue, einschließlich Sortierung und Filterung nach Kommentartypen (vgl. GitLab Docs, o. J.d, o. S.). Die Discussions API erlaubt das Erstellen, Ergänzen, Aktualisieren und Löschen von Issue-Threads und Notizen (vgl. GitLab Docs, o. J.e, o. S.). Diese APIs bilden die technische Grundlage dafür, dass Dev-Assist Ticketkontext lesen, Rückfragen oder Vorschläge posten und später Vorschläge übernehmen kann.

Technisch wird der Prototyp als Node.js-/TypeScript-/Express-Anwendung umgesetzt. Node.js ist eine freie, quelloffene und plattformübergreifende JavaScript-Laufzeitumgebung, mit der unter anderem Server, Webanwendungen, Kommandozeilenwerkzeuge und Skripte erstellt werden können (vgl. Node.js Docs, o. J.a, o. S.). Die HTTP-Dokumentation zeigt, dass Node.js HTTP-Server und -Clients bereitstellt und Anfragen sowie Antworten niedrigschwellig verarbeiten kann (vgl. Node.js Docs, o. J.b, o. S.). Für einen Webhook-Endpunkt ist diese Grundlage geeignet, weil GitLab Webhook-Ereignisse als HTTP-POST-Anfragen sendet.

Express ergänzt Node.js um ein minimalistisches und flexibles Webframework. Die Express-Dokumentation beschreibt Express als Framework für Webanwendungen und APIs mit HTTP-Hilfsmethoden und Middleware-Unterstützung (vgl. Express Docs, o. J., o. S.). Für Dev-Assist ist Express zweckmäßig, weil der Webhook-Endpunkt eingehende Requests entgegennehmen, Header prüfen, JSON-Payloads verarbeiten und schnell eine HTTP-Antwort an GitLab zurückgeben muss. Die eigentliche Analyse kann anschließend asynchron weiterlaufen, damit GitLab nicht auf lang laufende KI-

Verarbeitung warten muss.

TypeScript wird eingesetzt, um die dynamische JavaScript-Laufzeit um statische Typprüfung zu ergänzen. Das TypeScript-Handbook beschreibt TypeScript als statischen Typechecker für JavaScript-Programme, der vor der Ausführung prüft, ob Typen korrekt verwendet werden (vgl. TypeScript Docs, o. J., o. S.). Für den Prototyp ist das besonders relevant, weil GitLab-Payloads, API-Antworten und Agentenantworten strukturierte Daten enthalten, aber aus externen Quellen stammen. Typisierung unterstützt Wartbarkeit und Testbarkeit, ersetzt jedoch keine Laufzeitvalidierung. Deshalb muss die Anwendung zusätzlich prüfen, ob die vom Agenten gelieferten JSON-Strukturen tatsächlich dem erwarteten Format entsprechen.

Aus technischer Sicht ergibt sich damit ein Integrationsmuster: GitLab sendet ein Webhook-Ereignis an Express. Die Anwendung validiert grundlegende Eigenschaften des Requests, liest Issue- und Kommentar-Kontext über GitLab-Schnittstellen, startet eine Opencode-Agentenanalyse und schreibt das Ergebnis als Kommentar zurück. Bei einem Publish-Kommando wird ein zuvor erzeugter Vorschlag übernommen. Diese Kette verbindet Ereignisverarbeitung, API-Integration, KI-Analyse und kontrollierte Änderung des Tickets.

2.7 Risiken und Validierungsbedarf bei generativer KI und Webhooks

Der Einsatz generativer KI in einem Ticket-Workflow erzeugt Nutzen, aber auch Risiken. Das wichtigste fachliche Risiko besteht darin, dass ein Modell plausibel klingende, aber falsche oder nicht belegte Informationen erzeugt. Das National Institute of Standards and Technology beschreibt dieses Risiko als Confabulation beziehungsweise Hallucination und meint damit überzeugend dargestellte, aber fehlerhafte oder falsche Inhalte (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Huang et al. beschreiben Halluzinationen bei LLMs ähnlich als plausible, aber nicht faktische Inhalte, die die Zuverlässigkeit realer Informationssysteme beeinträchtigen können (vgl. Huang et al., 2023, S. 1 f.).

Für Dev-Assist ist dieses Risiko besonders relevant, weil Ticketvorschläge unmittelbare Auswirkungen auf Entwicklungsarbeit haben können. Wenn ein Assistent unbelegte Annahmen als Akzeptanzkriterien formuliert, kann daraus eine falsche Implementierung entstehen. Wenn er technische Ursachen erfindet, kann die Fehlersuche in eine falsche Richtung gelenkt werden. Wenn er fehlende Informationen nicht erkennt, wirkt das Ticket vollständiger, als es tatsächlich ist. Deshalb muss der Prototyp so konzipiert werden, dass er bei unzureichendem Kontext Rückfragen stellt und Vorschläge nicht automatisch ohne Freigabe übernimmt.

Ein zweites Risiko betrifft Prompt Injection. Liu et al. untersuchen Prompt-Injection-Angriffe gegen LLM-integrierte Anwendungen und zeigen, dass reale Anwendungen anfällig sein können; in ihrer Untersuchung waren 31 von 36 betrachteten Anwendungen gegenüber dem entwickelten Angriff anfällig (vgl. Liu et al., 2023, S. 1). OWASP beschreibt Prompt Injection als Schwachstelle, bei der Nutzereingaben das Verhalten oder die Ausgabe eines LLM in unbeabsichtigter Weise verändern können. Dabei un-

terscheidet OWASP direkte und indirekte Prompt Injection, wobei indirekte Angriffe über externe Quellen wie Dateien oder Webseiten erfolgen können (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection").

In einem GitLab-Assistenten können potenziell alle vom Modell gelesenen Inhalte Angriffs- oder Störsignale enthalten: Issue-Beschreibungen, Kommentare, Codeblöcke, Screenshots mit Text oder externe Links. Ein Kommentar könnte zum Beispiel versuchen, die Systemanweisungen des Assistenten zu überschreiben oder ihn dazu zu bringen, ungeprüfte Änderungen vorzunehmen. Daraus folgt, dass Ticketinhalte nicht als vertrauenswürdige Befehle behandelt werden dürfen. Sie sind Daten, die analysiert werden sollen. Entscheidungen über GitLab-Änderungen müssen in der Anwendung und nicht frei im Modell liegen.

Ein weiteres Sicherheitsproblem betrifft Webhooks selbst. Webhook-Endpunkte sind von außen erreichbare HTTP-Schnittstellen. Wenn ein Endpunkt unzureichend abgesichert ist, könnten fremde Systeme gefälschte Ereignisse senden. Nach aktueller GitLab-Dokumentation sollten neue Webhooks bevorzugt mit einem Signing Token abgesichert werden. GitLab berechnet damit eine HMAC-SHA256-Signatur über den Payload und übermittelt sie im Header `webhook-signature`, sodass der Empfänger Authentizität und Integrität der Anfrage prüfen kann. Ein Secret Token im Header `X-Gitlab-Token` bietet schwächere Garantien und wird für neue Webhooks nicht empfohlen (vgl. GitLab Docs, o. J.a, Abschnitt "Signing tokens"). Zusätzlich sollte der Zeitstempel `webhook-timestamp` geprüft werden, um Replay-Angriffe zu erschweren. Für den Prototyp ist daher wichtig, Signaturprüfung, relevante Header, erlaubte Ereignistypen und formal plausible Payloads gemeinsam zu berücksichtigen. Außerdem müssen Bot-Endlosschleifen verhindert werden: Wenn der Assistent auf seine eigenen Kommentare reagiert, kann eine unkontrollierte Folge von Webhook-Ereignissen entstehen.

Auch die Autonomie des Systems muss begrenzt werden. OWASP führt "Excessive Agency" als Risiko auf, wenn LLM-basierte Systeme über zu weitreichende Funktionalitäten, Berechtigungen oder Autonomie verfügen (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency"). Für Dev-Assist spricht dies für einen Vorschlagsmodus: Das System analysiert, fragt nach oder schlägt eine strukturierte Beschreibung vor. Die endgültige Übernahme erfolgt erst durch ein explizites Publish-Kommando. Dadurch bleibt die menschliche Kontrolle erhalten, und der Assistent kann in den Entwicklungsprozess integriert werden, ohne ungeprüft produktive Ticketinhalte zu verändern.

Validierungsbedarf besteht daher auf mehreren Ebenen. Eingehende Webhook-Payloads müssen formal plausibel sein. GitLab-API-Antworten müssen robust verarbeitet werden. Agentenantworten müssen als JSON parsebar sein und dem erwarteten Schema entsprechen. Fehlende oder widersprüchliche Informationen müssen als solche behandelt werden. Fehler beim Bildabruf oder bei der Vision-Verarbeitung dürfen nicht den gesamten Workflow blockieren. Logging muss nachvollziehbar machen, warum ein Ereignis ignoriert, analysiert oder veröffentlicht wurde. Diese Anforderungen bilden eine direkte Brücke zur späteren Anforderungsanalyse und Systemarchitektur.

2.8 Zwischenfazit und Anforderungen an Dev-Assist

Die Grundlagen zeigen, dass Software-Tickets eine zentrale Rolle als Arbeits- und Kommunikationsartefakte einnehmen. Anforderungen und Bug Reports müssen so dokumentiert sein, dass sie verstanden, umgesetzt und überprüft werden können. Forschung zu Bug Reports zeigt jedoch, dass relevante Informationen häufig fehlen oder unstrukturiert vorliegen. Besonders wichtig sind beobachtetes Verhalten, erwartetes Verhalten, Reproduktionsschritte, Kontextinformationen und klare Prüfkriterien. Gleichzeitig entstehen in GitLab-Issues Informationen nicht nur in der Beschreibung, sondern auch in Kommentaren, Diskussionen und Anhängen.

Large Language Models bieten eine Möglichkeit, solche natürlichsprachlichen und verteilten Informationen flexibel zu analysieren. Durch In-Context Learning und Prompt Engineering können sie angewiesen werden, Ticketinhalte zu strukturieren, Rückfragen zu formulieren oder Vorschläge zu erzeugen. LLM-basierte Agenten und Werkzeugnutzung zeigen darüber hinaus, dass solche Modelle in technische Workflows eingebunden werden können. Opencode stellt dafür im Prototyp die Agentenschnittstelle bereit, während GitLab über Webhooks und APIs den Ereignis- und Datenkontext liefert.

Aus den Risiken ergibt sich jedoch, dass Dev-Assist nicht als autonomer Entscheider konzipiert werden darf. Halluzinationen, Prompt Injection, fehlerhafte Payloads und zu weitreichende Handlungsrechte erfordern technische Grenzen. Der Assistent muss den Kontext analysieren, aber zwischen fehlender Information und gesicherter Information unterscheiden. Er muss Rückfragen stellen, wenn keine ausreichende Grundlage für einen Vorschlag besteht. Er muss Vorschläge in einem strukturierten Format liefern, das maschinell validiert werden kann. Und er darf Änderungen am Ticket nur über einen kontrollierten Publish-Prozess übernehmen.

Für die nachfolgende Anforderungsanalyse lassen sich daraus zentrale Anforderungen ableiten. Dev-Assist muss `@dev-assist` in Issues und Kommentaren erkennen, GitLab-Kontext über APIs abrufen, Issue-Beschreibung und Diskussion gemeinsam auswerten, Bild- und Screenshot-Kontext berücksichtigen, fehlende Informationen identifizieren, entweder Rückfragen oder strukturierte Vorschläge erzeugen, Agentenantworten zur Laufzeit validieren, eigene Bot-Ereignisse ignorieren und sicher auf fehlerhafte Eingaben reagieren. Diese Anforderungen werden im nächsten Kapitel konkretisiert und anschließend in eine Systemarchitektur überführt.

3 Anforderungsanalyse

Nachdem Kapitel 2 die fachlichen und technischen Grundlagen beschrieben hat, konkretisiert dieses Kapitel die Anforderungen an Dev-Assist. Im Mittelpunkt stehen die Anforderungen, die sich aus dem GitLab-basierten Ticketprozess, aus der Ticketqualitätsforschung und aus den Risiken generativer KI für den Prototyp ergeben. Die Anforderungsanalyse bildet damit die Brücke zwischen Grundlagen und Systemarchitektur.

Dev-Assist soll unstrukturierte oder unvollständige GitLab-Issues nicht automatisch in fertige Entwicklungsaufgaben verwandeln, sondern deren Aufbereitung unterstützen. Das System soll erkennen, wann es angesprochen wird, vorhandenen Ticketkontext auswerten, fehlende Informationen sichtbar machen, strukturierte Vorschläge erzeugen und die Übernahme nur nach einem expliziten Publish-Kommando ermöglichen. Die Anforderungen betreffen deshalb sowohl die Qualität der entstehenden Tickets als auch die robuste, nachvollziehbare und kontrollierte Integration in GitLab.

3.1 Vorgehen und Grundlagen der Anforderungsanalyse

Die Anforderungen werden aus vier Quellen abgeleitet. Erstens werden die Erkenntnisse aus Kapitel 2 zu Ticketqualität, Informationslücken und Risiken von Large Language Models herangezogen. Zweitens wird der bisherige manuelle GitLab-Prozess betrachtet. Drittens fließt die projektinterne Zielbeschreibung ein, nach der Dev-Assist über `@dev-assist` aktiviert wird, bei fehlendem Kontext Rückfragen stellt, bei ausreichendem Kontext einen Vorschlag erzeugt und diesen erst nach `@dev-assist publish` übernimmt. Viertens wird der aktuelle Prototyp berücksichtigt, um die Anforderungen mit den technischen Rahmenbedingungen abzugleichen.

Requirements Engineering umfasst nach der öffentlichen Katalogseite zu ISO/IEC/IEEE 29148 die systematische Behandlung von Anforderungen über den Lebenszyklus hinweg und beschreibt zugehörige Informationsprodukte und Prozesse (vgl. ISO/IEC/IEEE, 2018, o. S.). Für Dev-Assist bedeutet das, dass Anforderungen nicht nur Funktionen beschreiben, sondern auch benötigte Informationen, Verarbeitungsschritte und Systemgrenzen. Da GitLab-Issues in einem agilen Entwicklungskontext entstehen, ist außerdem die kollaborative Dimension relevant: Agile Requirements Engineering ist stark mit Kommunikation, gemeinsamer Klärung und kontinuierlicher Anpassung verbunden (vgl. Inayat et al., 2015, S. 915 ff.). Dev-Assist muss sich daher in bestehende Kommunikation einfügen, statt sie durch einen isolierten Analysevorgang zu ersetzen.

Die fachlichen Anforderungen orientieren sich an Arbeiten zu User Stories und Bug Reports. Lucassen et al. unterscheiden syntaktische, semantische und pragmatische Quali-

tätsaspekte von User Stories (vgl. Lucassen et al., 2016, S. 386 f.). Ein Ticketvorschlag darf deshalb nicht nur formal gegliedert sein, sondern muss inhaltlich verständlich, konsistent und für Entwicklerinnen und Entwickler nutzbar bleiben. Chaparro et al. zeigen für Bug Descriptions, dass besonders beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte häufig fehlen (vgl. Chaparro et al., 2017, S. 396 f.). Diese Befunde werden hier nicht als unmittelbarer Nachweis für alle Ticketarten verstanden. Für Dev-Assist wird daraus abgeleitet, dass GitLab-Tickets Ziel, Kontext, erwartetes Ergebnis, Akzeptanzkriterien, offene Fragen und relevante Randbedingungen möglichst explizit enthalten sollten.

Die technische Seite ergibt sich aus GitLab-Webhooks, GitLab-APIs und dem Prototyp-Stack. Webhooks ermöglichen externe Reaktionen auf GitLab-Ereignisse; Issues API, Notes API und Discussions API stellen Ticket- und Kommentarverlauf bereit (vgl. GitLab Docs, o. J.a, o. S.; GitLab Docs, o. J.c, o. S.; GitLab Docs, o. J.d, o. S.; GitLab Docs, o. J.e, o. S.). Dev-Assist muss daher Ereignisse entgegennehmen, relevante Ereignisse erkennen, Kontext abrufen und Ergebnisse nach GitLab zurückschreiben können. Gleichzeitig sind LLM-Ausgaben fehleranfällig. NIST beschreibt Halluzinationen beziehungsweise Confabulations als überzeugend oder sicher präsentierte, aber fehlerhafte oder falsche Inhalte (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Validierung, Rückfragen und menschliche Freigabe sind deshalb notwendige Bestandteile des Systementwurfs.

3.2 Beschreibung des bisherigen Prozesses

Der bisherige Prozess beginnt mit einem GitLab-Issue. Eine meldende Person beschreibt eine Anforderung, einen Fehler oder eine technische Aufgabe. Manche Tickets enthalten bereits Zielbeschreibung, Akzeptanzkriterien und Validierungshinweise; andere bestehen nur aus einer Idee, einem Screenshot, einer Fehlermeldung oder einem Gesprächsfragment. Damit ähnelt der Prozess den in der Forschung beschriebenen Bug-Tracking-Situationen, in denen meldende Personen häufig andere Informationen bereitstellen als Entwicklerinnen und Entwickler für die Bearbeitung benötigen (vgl. Bettenburg et al., 2008, S. 308 f.).

Fehlen Informationen, stellen Entwicklerinnen und Entwickler Rückfragen in den Kommentaren. Breu et al. zeigen, dass solche Rückfragen in Bug Reports unter anderem Reproduktionsschritte, Umgebung, Beispiele, Ausgaben und Screenshots betreffen können (vgl. Breu et al., 2010, S. 303). Übertragen auf GitLab-Issues bedeutet dies, dass die eigentliche Anforderung häufig nicht vollständig in der ursprünglichen Beschreibung steht, sondern schrittweise aus Titel, Beschreibung, Kommentaren, Antworten, Anhängen und Entscheidungen entsteht.

Dieser Ablauf erzeugt typische Probleme: Kontext ist verteilt, Wartezeiten entstehen durch manuelle Klärung, Rückfragen erfolgen unsystematisch, und nach der Klärung muss das Ticket oft erneut zusammengefasst und strukturiert werden. Ohne diesen Aufräumschritt bleibt das Ticket für Planung, Umsetzung und spätere Nachvollziehbarkeit schwer lesbar.

Dev-Assist soll den Prozess nicht ersetzen, sondern an dieser Stelle unterstützen. Das

System wird nur aktiv, wenn ein Issue oder Kommentar mit `@dev-assist` beginnt. Danach analysiert es den vorhandenen Kontext, stellt bei fehlenden Informationen Rückfragen oder erzeugt bei ausreichendem Kontext einen strukturierten Vorschlag. Erst nach `@dev-assist publish` wird dieser Vorschlag übernommen. Fachliche Verantwortung und Freigabe bleiben damit beim Menschen.

Der gewünschte Zielprozess umfasst fünf Schritte:

1. Eine Person erstellt oder aktualisiert ein GitLab-Issue und spricht Dev-Assist mit `@dev-assist` an.
2. Dev-Assist prüft das Ereignis, liest den verfügbaren Issue- und Kommentar-Kontext und startet die Analyse.
3. Bei unzureichendem Kontext stellt Dev-Assist gezielte Rückfragen im Kommentarverlauf.
4. Bei ausreichendem Kontext erzeugt Dev-Assist einen strukturierten Ticketvorschlag mit Titel, Ziel, Umfang, Anforderungen, Akzeptanzkriterien, offenen Fragen, Risiken und Validierungsschritten.
5. Nach `@dev-assist publish` werden Dev-Assist-bezogene Kommentare bereinigt und Titel sowie Beschreibung des Issues durch den strukturierten Vorschlag ersetzt.

Dev-Assist wird damit als Prozesswerkzeug verstanden. Es verbessert die Qualität des Ticketartefakts, ohne GitLab als Arbeitsort zu verlassen oder eine zusätzliche Benutzeroberfläche einzuführen.

3.3 Zielgruppen des Systems

Dev-Assist richtet sich an alle Rollen, die am Entstehen und Nutzen von GitLab-Tickets beteiligt sind. Ihre Bedarfe unterscheiden sich und führen zu unterschiedlichen Anforderungen an Rückfragen, Strukturierung, Prüfbarkeit und Betrieb.

Zielgruppe	Bedarf im Prozess	Konsequenz für Dev-Assist
Meldende Personen und fachliche Stakeholder	Niedrige Einstiegshürde beim Formulieren von Anforderungen oder Fehlern	Dev-Assist darf keine perfekte Erstbeschreibung voraussetzen und muss verständliche Rückfragen stellen.
Product Owner und Projektverantwortliche	Besser strukturierte Tickets, klare Abgrenzung und nachvollziehbare Akzeptanzkriterien	Der Vorschlag muss Ziel, Nutzen, Scope, Out-of-Scope und Definition of Done sichtbar machen.

Zielgruppe	Bedarf im Prozess	Konsequenz für Dev-Assist
Entwicklerinnen und Entwickler	Bearbeitbare Aufgaben mit ausreichend Kontext und prüfbaren Kriterien	Der erzeugte Tickettext muss Umsetzung, Validierung und offene Annahmen klar trennen.
Qualitätssicherung	Prüfbarkeit der Umsetzung und klare erwartete Ergebnisse	Akzeptanzkriterien und Validierungsschritte müssen explizit aufgeführt werden.
Administratorinnen und Betreiber	Sicherer und wartbarer Betrieb der Webhook-Anwendung	Konfiguration, Logging, Fehlerverhalten und Signaturprüfung müssen nachvollziehbar sein.

Für meldende Personen ist wichtig, dass Dev-Assist nicht mit technischen Detailfragen beginnt. Die projektinterne Ausrichtung sieht vor, dass der Assistent nach Anforderungen, Nutzerbedarf, Akzeptanzkriterien, Scope und Erfolgskriterien fragt, nicht nach konkreten Dateien oder Implementierungsdetails. Diese Abgrenzung ist fachlich sinnvoll, weil meldende Personen bestimmte für Entwickler hilfreiche Informationen oft nicht ohne Weiteres ermitteln oder verwertbar bereitstellen können (vgl. Bettenburg et al., 2008, S. 308 f.).

Für Entwicklerinnen und Entwickler zählt die spätere Nutzbarkeit. Der Vorschlag muss beschreiben, was gebaut oder geändert werden soll, ohne unbelegte Implementierungsentscheidungen zu treffen. Technische Hinweise sind nur aufzunehmen, wenn sie aus dem Ticketkontext hervorgehen oder ausdrücklich als Randbedingung genannt wurden. Für Product Owner und Qualitätssicherung sind Ziel, erwartetes Verhalten und Akzeptanzkriterien zentral, da nur so Priorisierung, Umsetzung und Abnahme möglich sind. Diese Logik passt auch zu Chaparro et al., die erwartetes Verhalten und Reproduktionsschritte als wichtige Bestandteile von Bug Descriptions beschreiben (vgl. Chaparro et al., 2017, S. 396 f.).

Für Betreiberinnen und Betreiber muss Dev-Assist kontrollierbar bleiben. Das System verarbeitet externe Webhook-Anfragen und nutzt generative KI. Daher müssen Secrets konfigurierbar sein, Ereignisse protokolliert werden, Signaturprüfungen für produktive Umgebungen erzwingbar sein und Fehler ohne unkontrollierte Issue-Änderungen behandelt werden.

3.4 Fachliche Anforderungen an die Ticketqualität

Das zentrale fachliche Ziel ist ein entwicklungsfähiges Ticket. In dieser Arbeit gilt ein Ticket als entwicklungsfähig, wenn eine Entwicklerin oder ein Entwickler die Aufgabe verstehen, umsetzen und überprüfen kann, ohne wesentliche Informationen aus verstreuten Kommentaren rekonstruieren zu müssen.

Dafür muss der Titel den Gegenstand knapp beschreiben, das Ziel den Nutzen oder das zu lösende Problem benennen und der Umfang klar abgegrenzt sein. Scope und Out-of-

Scope verhindern, dass Tickets zu groß werden oder verschiedene Themen vermischen. Funktionale Anforderungen und Akzeptanzkriterien müssen explizit formuliert werden. Lucassen et al. betonen, dass User-Story-Qualität nicht nur an der Form, sondern auch an Bedeutung und Nutzbarkeit gemessen werden muss (vgl. Lucassen et al., 2016, S. 386 f.). Eine bloße sprachliche Umformulierung reicht daher nicht aus; Dev-Assist soll vorhandene Inhalte in prüfbare Aussagen überführen.

Offene Fragen, Risiken und Annahmen müssen sichtbar bleiben. Wenn Informationen fehlen, darf das System sie nicht durch scheinbar sichere Aussagen ersetzen. Das ist besonders wichtig, weil LLMs überzeugend wirkende, aber fehlerhafte Inhalte erzeugen können (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Unsichere Ableitungen müssen daher als Annahmen oder offene Punkte erkennbar sein.

Aus diesen Qualitätszielen ergibt sich folgende Mindeststruktur:

Bestandteil	Zweck
Zusammenfassung	Kurze Einordnung des Ticketinhalts und der Quelle des Vorschlags
Titel	Knappes, suchbares und handlungsorientiertes Issue-Thema
Ziel	Beschreibung des angestrebten Ergebnisses oder Nutzens
Scope	Inhalte, die ausdrücklich Teil des Tickets sind
Out-of-Scope	Inhalte, die nicht Teil des Tickets sind
User Stories	Rollenbezogene Beschreibung des erwarteten Nutzens, sofern sinnvoll ableitbar
Funktionale Anforderungen	Konkrete Verhaltensanforderungen an das System
Technische Hinweise	Nur explizit genannte technische Randbedingungen, keine erfundenen Implementierungsdetails
Umsetzungsschritte	Grobe, entwicklungsorientierte Arbeitsschritte ohne unnötige Tiefendetails
Definition of Done	Bedingungen, unter denen das Ticket als abgeschlossen gelten kann
Akzeptanzkriterien	Prüfpunkte für Abnahme und Qualitätssicherung
Offene Fragen	Fehlende Informationen, die nicht sicher abgeleitet werden können
Risiken und Annahmen	Unsicherheiten, Abhängigkeiten und mögliche Fehlinterpretationen
Validierungsschritte	Vorschläge, wie Umsetzung oder Fehlerbehebung geprüft werden kann

Die Struktur ist bewusst umfangreicher als eine minimale User Story. Sie soll Tickets nicht künstlich aufblähen, sondern sichtbar machen, welche Informationen bereits vorhanden sind und welche fehlen. Wenn ein Abschnitt nicht sinnvoll gefüllt werden kann, muss Dev-Assist die Lücke benennen, statt leere Qualität zu simulieren.

3.5 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben, was Dev-Assist leisten muss. Die Nummerierung FA-1 bis FA-11 dient der Nachverfolgbarkeit in Konzeption, Implementierung und Evaluation.

FA-1: Explizite Aktivierung durch @dev-assist. Dev-Assist darf nur reagieren, wenn der erste inhaltliche Text eines Issues oder Kommentars mit `@dev-assist` beginnt. Führende Leerzeichen und einfache Markdown-Formatierungen wie Überschriften, Listenmarker oder Fettschreibung sollen toleriert werden. Dadurch analysiert das System keine beliebige GitLab-Kommunikation und reagiert nicht auf zufällige Erwähnungen im Fließtext.

FA-2: Verarbeitung relevanter GitLab-Ereignisse. Das System muss Work-item-/Issue-Ereignisse und Kommentarereignisse aus GitLab-Webhooks entgegennehmen können. Relevant ist ein Ereignis, wenn Issue-Beschreibung oder Kommentar Dev-Assist am Anfang ansprechen. GitLab-Webhooks liefern Ereignisse als HTTP-Anfragen an externe Anwendungen; GitLab dokumentiert dafür unter anderem Work-item-/Issue-bezogene Ereignisse und Comment Events (vgl. GitLab Docs, o. J.a, o. S.; GitLab Docs, o. J.b, Abschnitte "Work item events" und "Comment events"). Dev-Assist muss diese Payloads in ein internes, einheitliches Format überführen.

FA-3: Unterscheidung zwischen Analyse- und Publish-Kommando. Nach der Mention muss Dev-Assist zwischen Analyseanforderung und Publish-Kommando unterscheiden. Beginnt der restliche Kommentar mit `publish`, wird keine neue Analyse gestartet, sondern der zuletzt erzeugte strukturierte Kontext übernommen. Alle anderen Aktivierungen lösen die Analyse aus.

FA-4: Abruf des relevanten Ticketkontexts. Die Webhook-Payload reicht für die Analyse nicht immer aus. Dev-Assist muss deshalb, soweit möglich, das aktuelle Issue und die zugehörigen Kommentare über GitLab abrufen. Die Issues API stellt Issue-Daten bereit, die Notes API den Kommentarverlauf (vgl. GitLab Docs, o. J.c, o. S.; GitLab Docs, o. J.d, o. S.). Der Kommentarverlauf ist wichtig, weil Informationen häufig nachträglich ergänzt werden.

FA-5: Analyse von Beschreibung und Kommentaren. Dev-Assist muss Titel, Beschreibung, Kommentare und gegebenenfalls zusätzlichen Rohtext gemeinsam auswerten. Das System soll vorhandene und fehlende Informationen erkennen und zwischen Anforderungen, Akzeptanzkriterien, technischen Randbedingungen, offenen Fragen und Annahmen unterscheiden. Diese Anforderung folgt aus dem Charakter von Tickets als verteilte Arbeitsartefakte.

FA-6: Rückfragen bei fehlenden Informationen. Wenn wesentliche Informationen fehlen, muss Dev-Assist gezielte Rückfragen formulieren. Sie sollen fachliche Ziele, Nutzerbedarf, Scope, Akzeptanzkriterien, Randfälle und Erfolgskriterien betreffen, nicht interne Implementierungsdetails. Die Fragen müssen spezifisch genug sein, um den Ticketkontext tatsächlich zu verbessern. Diese Anforderung knüpft an die Forschung zu fehlenden Informationen in Bug Reports an (vgl. Breu et al., 2010, S. 303; Chaparro et al., 2017, S. 396 f.).

FA-7: Erzeugung strukturierter Ticketvorschläge. Wenn Hauptzweck, zentra-

le Anforderungen und Umfang ausreichend klar sind, muss Dev-Assist einen strukturierten Vorschlag nach Abschnitt 3.4 erzeugen. Der Vorschlag muss als neue Issue-Beschreibung nutzbar sein und darf ungesicherte Informationen nicht als Tatsachen darstellen. Offene Punkte werden als offene Fragen oder Annahmen markiert. Dieser Modus knüpft an Qualitätshilfen wie BEE an, das Bug Reports anhand zentraler Bestandteile wie beobachtetem Verhalten, erwartetem Verhalten und Reproduktionsschritten strukturiert; die breitere Dev-Assist-Struktur ist eine eigene Anforderung dieser Arbeit (vgl. Song/Chaparro, 2020, S. 1551 f.).

FA-8: Maschinenlesbares Analyseformat. Die KI-Ausgabe muss in einem definierten JSON-Format erfolgen. Das Format muss Felder für Zusammenfassung, Quellenbasis, Umsetzungsticket, Akzeptanzkriterien, technische Hinweise, offene Fragen, Risiken und Validierungsschritte enthalten. Das System muss die Antwort parsen und strukturell prüfen, bevor daraus ein Kommentar oder Kontextdokument entsteht.

FA-9: Veröffentlichung als GitLab-Kommentar. Das Analyseergebnis muss in GitLab sichtbar werden. Bei fehlendem Kontext postet Dev-Assist Rückfragen, bei ausreichendem Kontext einen vollständigen strukturierten Vorschlag. Der Kommentar muss deutlich machen, dass die Übernahme erst durch `@dev-assist publish` erfolgt.

FA-10: Persistenz des strukturierten Kontexts. Nach der Analyse muss Dev-Assist den strukturierten Kontext lokal in einem reproduzierbaren Pfad ablegen, etwa unter `.dev-assist/issues/<projectId>/<issueId>/context.md`. Verfügbare Titel oder Metadaten sollen zusätzlich maschinenlesbar gespeichert werden. Diese Kontextdateien bilden die Grundlage für den Publish-Schritt und mögliche spätere Entwicklungsprozesse.

FA-11: Kontrollierte Übernahme durch Publish. Das Publish-Kommando muss den zuvor erzeugten Kontext lesen, Dev-Assist-bezogene Kommentare identifizieren, diese bereinigen und anschließend Titel sowie Beschreibung des GitLab-Issues aktualisieren. Publish darf nur nach erfolgreicher Analyse erfolgen. Die Anforderung begrenzt die Systemautonomie und passt zu OWASPs Risiko "Excessive Agency", bei dem LLM-basierte Systeme zu weitreichende Berechtigungen oder Handlungsspielräume erhalten (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency").

3.6 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen beschreiben Qualitätsmerkmale für Betrieb, Wartung und Vertrauen. Da Dev-Assist Webhooks, GitLab-API-Zugriffe und generative KI verbindet, sind diese Anforderungen besonders relevant.

NFA-1: Wartbarkeit durch klare Modulgrenzen. Webhook-Verarbeitung, GitLab-Anbindung, Mention-Erkennung, KI-Analyse, Formatierung, Kontextpersistenz und Publish-Logik müssen getrennt nachvollziehbar sein. Klare Modulgrenzen erleichtern Änderungen, etwa den Austausch des KI-Providers oder die Erweiterung der GitLab-Operationen.

NFA-2: Testbarkeit ohne echte GitLab- oder KI-Abhängigkeit. Dev-Assist muss lokal testbar sein, ohne reale GitLab-Projekte oder kostenpflichtige KI-Aufrufe zu

benötigen. Dafür braucht das System einen Mock-Modus für KI-Analysen und isolierbare Funktionen für Parser, Mention-Erkennung, Formatierung, Cleanup und Webhook-Authentifizierung. Wegen der Nichtdeterministik von LLM-Ausgaben müssen die deterministischen Systemteile besonders gut abgesichert werden.

NFA-3: Robustheit gegenüber unvollständigem Kontext und externen Fehlern. Das System muss mit fehlgeschlagenen GitLab-API-Aufrufen, unvollständigen Payloads, fehlenden Kommentaren oder ungültigen KI-Antworten umgehen können. Fehler sollen protokolliert und, soweit sinnvoll, mit Fallback-Daten behandelt werden. Wenn eine Analyse vollständig fehlschlägt, darf kein fehlerhafter Publish-Kontext entstehen.

NFA-4: Nachvollziehbarkeit durch Logging. Relevante Verarbeitungsschritte müssen über Konsolenausgaben nachvollziehbar sein: eingehende Requests, erkannte und ignorierte Ereignisse, Analysebeginn und -ende, Fehler, gepostete Kommentare, Kontextdateien, gelöschte Kommentare und aktualisierte Issues. Da keine separaten Logdateien vorgesehen sind, ist die Konsole die zentrale Beobachtungsquelle. Logging muss konkret sein, darf aber keine Secrets ausgeben.

NFA-5: Begrenzung von KI-Risiken. LLM-Ausgaben dürfen nicht ungeprüft zu produktiven Änderungen werden. Halluzinationen und Confabulations sind für generative KI ein bekanntes Risiko (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Daher müssen KI-Antworten strukturell validiert, offene Fragen erhalten und Vorschläge erst nach expliziter Freigabe übernommen werden. Der Prompt muss zudem klarstellen, dass Ticketinhalte Daten und keine Systemanweisungen sind, da OWASP Prompt Injection als Risiko beschreibt, bei dem Eingaben das Verhalten eines LLM unbeabsichtigt verändern können (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection").

NFA-6: Sicherheit der Webhook-Schnittstelle. Dev-Assist muss eine Signatur- oder Tokenprüfung für Webhook-Anfragen unterstützen und produktiv erzwingbar machen. Lokale Entwicklung soll auch ohne eingerichtetes GitLab-Secret möglich bleiben. Für Tests kann ein toleranter Modus sinnvoll sein; produktiv muss die Integrität der Anfrage abgesichert werden.

NFA-7: Vermeidung von Bot-Endlosschleifen. Dev-Assist muss verhindern, dass eigene Kommentare neue Analysen auslösen. Dazu verarbeitet das System nur führende `@dev-assist`-Mentions, generierte Vorschläge beginnen nicht mit dieser Mention, und doppelte Ereignisse müssen innerhalb eines kurzen Zeitfensters ignoriert werden können.

NFA-8: Schnelle Webhook-Antwort und asynchrone Verarbeitung. GitLab-Webhooks sollten zeitnah mit einem erfolgreichen HTTP-Status beantwortet werden, während KI-Analysen länger dauern können (vgl. GitLab Docs, o. J.a, o. S.). Dev-Assist muss Webhook-Anfragen daher schnell mit einem erfolgreichen 2xx-Status, etwa 200 oder 201, beantworten und die Verarbeitung asynchron oder nachgelagert ausführen. So werden Timeouts und erneut gesendete Ereignisse vermieden.

NFA-9: Konfigurierbarkeit des Betriebs. Port, Log-Level, Mention-String, GitLab-Basis-URL, GitLab-Authentifizierung, KI-Provider, Modell, Timeout, Kontextverzeichnis und Signaturpflicht müssen über Umgebungsvariablen steuerbar sein. Secrets gehö-

ren in die `.env` und nicht in den Code. Diese Anforderung unterstützt Mock-Betrieb, `glab`, PAT-Fallback und spätere Modellerweiterungen.

NFA-10: Prozessintegrierte Bedienbarkeit. Dev-Assist soll keine zusätzliche grafische Oberfläche benötigen. Die Bedienung erfolgt über GitLab-Issues und Kommentare, sodass der Arbeitsfluss vertraut bleibt. Dashboards, Metriken oder separate Administrationsoberflächen sind mögliche Erweiterungen, aber nicht Teil des Kernumfangs.

3.7 Abgrenzung der Anforderungen

Dev-Assist soll nicht zu einem allgemeinen Entwicklungsagenten ausgeweitet werden. Die Arbeit konzentriert sich auf die Verbesserung von Software-Tickets.

Erstens führt Dev-Assist keine autonome Implementierung durch. Das System erzeugt strukturierte Tickets, Rückfragen und Vorschläge, schreibt aber keinen Produktivcode, erstellt keine Merge Requests und entscheidet nicht eigenständig über technische Umsetzung. Zweitens ersetzt Dev-Assist keine fachliche Priorisierung. Ob ein Ticket wichtig ist, wann es umgesetzt wird und welche Produktentscheidung dahintersteht, bleibt Aufgabe der verantwortlichen Personen. Drittens ersetzt der Assistent keine menschliche Prüfung: Der Publish-Schritt ist bewusst explizit.

Viertens bewertet Dev-Assist nicht die gesamte Qualität eines Projekts. Es geht nicht um Codequalität, Architekturqualität oder Produktmetriken, sondern um Struktur und Nutzbarkeit einzelner Tickets. Fünftens garantiert das System keine vollständige semantische Wahrheit. LLM-Ausgaben bleiben Vorschläge; Unsicherheiten müssen sichtbar bleiben, und die Verantwortung verbleibt beim Team.

Sechstens ist eine umfassende Bild- oder Screenshot-Interpretation nicht Kern der Anforderungsanalyse. Screenshots können Hinweise liefern und spätere Erweiterungen ermöglichen, für den Kernprozess steht jedoch die Strukturierung vorhandener textueller Informationen aus Beschreibung und Kommentaren im Vordergrund.

3.8 Priorisierung der Anforderungen

Für den Prototyp werden die Anforderungen nach ihrer Bedeutung für den Kernworkflow priorisiert. Muss-Anforderungen sind notwendig, Soll-Anforderungen verbessern Qualität, Betrieb oder Erweiterbarkeit, und Kann-Anforderungen sind sinnvoll, aber nicht Voraussetzung für den Kernnachweis.

Priorität	Anforderungen	Begründung
Muss	FA-1 bis FA-11, NFA-1 bis NFA-10	Ohne expliziten Trigger, Analyse, Rückfragen, Vorschlag, Persistenz, Publish, Validierung, Logging, Konfiguration, Sicherheitsgrenzen und Schleifenvermeidung ist der Zielprozess nicht belastbar erfüllt.
Soll	Erweiterte Behandlung von Bild- und Screenshot-Kontext, zusätzliche GitLab-Metadaten, ausführlichere Fehlerdiagnosen	Diese Punkte verbessern den praktischen Nutzen, sind aber nicht erforderlich, um den Kernworkflow nachzuweisen.
Kann	Dashboard, quantitative Qualitätsmetriken, Rollen- und Rechtekonzept über GitLab hinaus	Diese Erweiterungen sind fachlich interessant, aber für den prototypischen Nachweis nicht erforderlich.

Die Priorisierung spiegelt die Zielsetzung wider: Dev-Assist soll zeigen, dass ein KI-gestützter GitLab-Assistent Ticketinformationen in einem kontrollierten Workflow verbessern kann. Entscheidend sind daher Trigger-Erkennung, Kontextanalyse, strukturierte Ausgabe und menschlich kontrollierte Übernahme, nicht zusätzliche Oberflächen oder umfassende Produktivbetriebsfunktionen.

3.9 Zwischenfazit

Die Anforderungsanalyse zeigt, dass Dev-Assist zwei Ebenen verbinden muss. Fachlich soll das System unvollständige oder unstrukturierte GitLab-Issues in besser nutzbare Entwicklungsartefakte überführen. Dafür muss es fehlende Informationen erkennen, Rückfragen stellen, strukturierte Vorschläge erzeugen und Annahmen sichtbar machen. Technisch muss Dev-Assist GitLab-Ereignisse verarbeiten, Ticketkontext abrufen, KI-Ausgaben validieren, Ergebnisse zurückschreiben und den Publish-Schritt kontrolliert ausführen.

Besonders wichtig ist die Begrenzung der Systemautonomie. Die Forschung zu Ticketqualität begründet den Nutzen strukturierter Anforderungen, Akzeptanzkriterien und Rückfragen. Die Risiken generativer KI begründen, warum Dev-Assist keine Änderungen ohne Freigabe vornehmen darf. Daraus ergibt sich der zentrale Entwurfsansatz: Dev-Assist ist ein Assistenzsystem zur Ticketaufbereitung, kein autonomer Entscheider.

Die Anforderungen dieses Kapitels bilden die Grundlage für die Systemarchitektur im

nächsten Kapitel. Dort wird beschrieben, wie Webhook-Handler, GitLab-API-Anbindung, Agenten-Analyse, Validierung, Kontextdateien, Kommentarverarbeitung und Publish-Funktion zusammenspielen.

Quellen

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, S. 308-318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports. Improving Cooperation Between Developers and Users“, in Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work, S. 301-310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Brown, Tom B. et al. 2020. „Language Models are Few-Shot Learners“, in Advances in Neural Information Processing Systems 33, S. 1877-1901. <https://arxiv.org/abs/2005.14165>.

Chaparro, Oscar; Bernal-Cárdenas, Carlos; Lu, Jing; Moran, Kevin; Marcus, Andrian; Di Penta, Massimiliano; Poshypanyk, Denys; Ng, Vincent 2019. „Assessing the Quality of the Steps to Reproduce in Bug Reports“, in Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, S. 152-163. New York: ACM. <https://doi.org/10.1145/3338906.3338947>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, S. 396-407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Express Docs o. J. Express.js. Node.js web application framework. <https://expressjs.com/en/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.a. Webhooks. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.b. Webhook events. https://docs.gitlab.com/user/project/integrations/webhook_events/ (Zugriff vom 25.06.2026).

GitLab Docs o. J.c. Issues API. <https://docs.gitlab.com/api/issues/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.d. Notes API. <https://docs.gitlab.com/api/notes/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.e. Discussions API. <https://docs.gitlab.com/api/discussions/> (Zugriff vom 25.06.2026).

Huang, Lei et al. 2023. „A Survey on Hallucination in Large Language Models. Principles, Taxonomy, Challenges, and Open Questions“. <https://arxiv.org/abs/2311.05232>.

Inayat, Irum; Salim, Siti Salwah; Marczak, Sabrina; Daneva, Maya; Shamshirband, Shahaboddin 2015. „A systematic literature review on agile requirements engineering practices and challenges“, in Computers in Human Behavior 51, S. 915-929. <https://doi.org/10.1016/j.chb.2014.10.046>.

ISO/IEC 2023. ISO/IEC 25010:2023. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model. Öffentliche Katalogseite. <https://www.iso.org/standard/78176.html>.

ISO/IEC/IEEE 2018. ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering. Öffentliche Katalogseite. <https://www.iso.org/standard/72089.html>.

Liu, Yi et al. 2023. „Prompt Injection attack against LLM-integrated Applications“. <https://arxiv.org/abs/2306.05499>.

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving agile requirements. The Quality User Story framework and tool“, in Requirements Engineering 21, 3, S. 383-403. <https://doi.org/10.1007/s00766-016-0250-x>.

National Institute of Standards and Technology 2024. Artificial Intelligence Risk Management Framework. Generative Artificial Intelligence Profile. NIST AI 600-1. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

Node.js Docs o. J.a. Node.js. <https://nodejs.org/en> (Zugriff vom 25.06.2026).

Node.js Docs o. J.b. HTTP. <https://nodejs.org/api/http.html> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.a. Agents. <https://opencode.ai/docs/agents/> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.b. SDK. <https://opencode.ai/docs/sdk/> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.c. Tools. <https://opencode.ai/docs/tools/> (Zugriff vom 25.06.2026).

OWASP 2025a. LLM01:2025 Prompt Injection. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Zugriff vom 25.06.2026).

OWASP 2025b. LLM06:2025 Excessive Agency. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/> (Zugriff vom 25.06.2026).

Schick, Timo et al. 2023. „Toolformer. Language Models Can Teach Themselves to Use Tools“, in Advances in Neural Information Processing Systems 36. <https://arxiv.org/abs/2302.04761>.

Song, Yang; Chaparro, Oscar 2020. „BEE. A Tool for Structuring and Analyzing Bug Reports“, in Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, S. 1551-1555. New York: ACM. <https://doi.org/10.1145/3368089.3417928>.

TypeScript Docs o. J. The TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html> (Zugriff vom 25.06.2026).

Vaswani, Ashish et al. 2017. „Attention Is All You Need“, in Advances in Neural Information Processing Systems 30. <https://papers.neurips.cc/paper/7181-attention-is-all-you-need>.

Xi, Zhiheng et al. 2023. „The Rise and Potential of Large Language Model Based Agents. A Survey“. <https://arxiv.org/abs/2309.07864>.

Yao, Shunyu et al. 2023. „ReAct. Synergizing Reasoning and Acting in Language Models“, in International Conference on Learning Representations 2023. https://openreview.net/forum?id=WE_vluYUL-X.

Zhao, Wayne Xin et al. 2023. „A Survey of Large Language Models“. <https://arxiv.org/abs/2303.18223>.

Projektinterne Arbeitsgrundlagen: `descriptions/project_description.txt`, `README.md`, `src/services/gitlab/parser.ts`, `src/services/gitlab/mention.ts`, `src/services/gitlab/commands.ts`, `src/services/gitlab/auth.ts`, `src/services/gitlab/cleanup.ts`, `src/services/ai/instructions.ts`, `src/services/ai/schema.ts`, `src/services/ai/formatter.ts`, `src/services/processing/processor.ts`, `src/services/processing/publisher.ts`, `src/services/context/writer.ts`, `src/routes/gitlabWebhooks.ts`, `src/routes/issues.ts`, `tests/gitlab.test.ts`, `tests/auth.test.ts`, `tests/cleanup.test.ts` und `tests/formatter.test.ts`.